

BookMyShow — Scaling, Reliability and Observability

1. High Concurrency & Scalability Techniques

Horizontal Scaling of Application Servers

- All core services (`SearchService` , `BookingService` , `PaymentService` , `NotificationService`) are completely stateless.
- Services run as containerized workloads behind load balancers. During traffic surges (such as ticket drops for blockbuster releases), auto-scaling policies launch additional instances automatically based on CPU utilization and request throughput.

Load Balancing Strategies

- **Round Robin:** Distributes read queries evenly across stateless `SearchService` instances where request payloads are uniform in processing cost.
- **Least Connections:** Routes transactional requests (`POST /bookings` and checkout flows) to instances handling the fewest active connections, preventing individual nodes from becoming saturated during database transactions.
- **Weighted:** Leveraged during canary releases to route a small proportion of user traffic (e.g., 5%) to newly deployed service versions to validate stability before full traffic cutover.

Database Sharding

- **Sharding Key:** The relational database is sharded geographically by `city` (or `theater`).
- **Rationale:** Movie discovery and seat bookings are strictly local; a user booking a show in Delhi never competes for seats or queries schedules in Mumbai. Sharding by city isolates transactional loads, ensuring heavy booking spikes in one metro do not degrade performance in another.

Replication & Fault Tolerance

- Primary database instances replicate data asynchronously to multiple standby read replicas distributed across distinct availability zones.
- If the primary node experiences hardware failure or network unreachability, an automated health probe triggers promotion of a healthy replica to primary, minimizing downtime.

Read/Write Separation

- **Writes:** All transactional writes (seat locking, booking creation, payment updates) are routed strictly to the primary database to ensure ACID compliance.
- **Reads:** Read-intensive traffic (browsing movie catalogs, viewing theater schedules, loading seat layouts) is directed to read replicas, offloading over 80% of total database traffic.

2. Caching Strategy & Invalidation

What Data to Cache (Redis)

- **Static / Semi-Static Data:** Movie metadata, cinema descriptions, and auditorium seat layouts. This data rarely changes and is cached with long TTLs (12 to 24 hours).
- **Dynamic / Volatile Data:** Active show availability and temporary seat lock tokens. These use short TTLs (5 to 8 minutes) to match the reservation hold window.

Cache Invalidation Approaches

- **TTL-Based Invalidation:** Temporary seat holds expire automatically via Redis key expiration without requiring explicit deletion.
- **Event-Driven Invalidation:** When a theater administrator updates a show schedule or changes ticket pricing, an update event is published to invalidate or refresh the specific Redis cache keys immediately.

3. Retry Policies & Dead-Letter Queues (DLQ)

Exponential Backoff

For asynchronous operations (such as dispatching notification emails/SMS or receiving payment gateway webhooks), transient network errors trigger retries with exponential delays (e.g., 1s, 2s, 4s) accompanied by random jitter. This prevents retry storms from overwhelming recovering downstream systems.

Dead-Letter Queue (DLQ)

If an event fails after a maximum retry threshold (e.g., 3 attempts), the message is moved to a dedicated Dead-Letter Queue in Kafka (e.g., `notifications-dlq`). This prevents malformed messages from blocking the primary consumer partition, preserves failed events for debugging, and enables manual replay once underlying dependencies recover.

4. Key Observability Metrics & Logging

- **API Latency and Throughput:** Tracks requests-per-second (RPS) alongside p50, p95, and p99 response times for critical endpoints (`GET /movies` , `POST /bookings`). Elevated p95/p99 numbers reveal connection pool exhaustion or slow database queries early.
- **Booking Success Rate:** Calculates the ratio of confirmed bookings to initiated checkouts. A sharp decline indicates seat lock conflicts, database lock contention, or transaction timeouts.
- **Payment Failures per Minute:** Tracks transaction failures, timeouts, and declines reported by third-party payment gateways. Spikes immediately trigger alerts to route checkouts to a secondary payment provider.
- **Cache Hit/Miss Ratio:** Measures the percentage of catalog and show queries served directly from Redis. A falling hit ratio warns of cache stampedes or key evictions threatening to overwhelm PostgreSQL read replicas.
- **Centralized Structured Logging:** Microservices emit structured JSON logs tagged with a unified `correlationId` (or `requestId`). This allows engineers to trace a user's transaction end-to-end across the API Gateway, BookingService, and PaymentService in a single log query.

5. Observability Tooling Stack

- **Prometheus:** Periodically scrapes and stores operational time-series metrics from microservices, Redis, Kafka, and database exporters.
- **Grafana:** Displays unified real-time dashboards for service health, request volume spikes, error rates, and latency percentiles.
- **ELK Stack (Elasticsearch, Logstash, Kibana):** Ingests, indexes, and indexes application logs across all distributed components to support fast debugging and root-cause analysis during incidents.

6. Supporting Service Level Objectives (SLOs)

- **Latency SLO:**

- *Target:* 95th percentile (p95) response time for `GET /movies` remains under **200 ms**.
- *Enforcement:* If p95 latency exceeds 200 ms over a 3-minute rolling window, an alert notifies the on-call engineer to check Redis cache hit rates or scale application instances.

- **Availability SLO:**

- *Target:* **99.9%** successful completion rate for end-to-end ticket checkout requests.
- *Enforcement:* Monitoring payment failures per minute alongside checkout errors isolates whether degraded availability is caused by internal database deadlocks or external payment partner downtime.